

TOPIC 2

Experiment 1

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        min_index = i
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
    return arr
```

```
# Test Cases
```

```
test_cases = [
```

```
    [],
```

```
    [1],
```

```
    [7, 7, 7, 7],
```

```
    [-5, -1, -3, -2, -4]
```

```
]
```

```
for arr in test_cases:
```

```
    print("Input :", arr)
```

```
    print("Output:", selection_sort(arr.copy()))
```

```
    print()
```

Sample Input:

Test Case 1:

```
[]
```

Test Case 2:

```
[1]
```

Test Case 3:

```
[7, 7, 7, 7]
```

Test Case 4:

```
[-5, -1, -3, -2, -4]
```

Sample Output:

Test Case 1:

[]

Test Case 2:

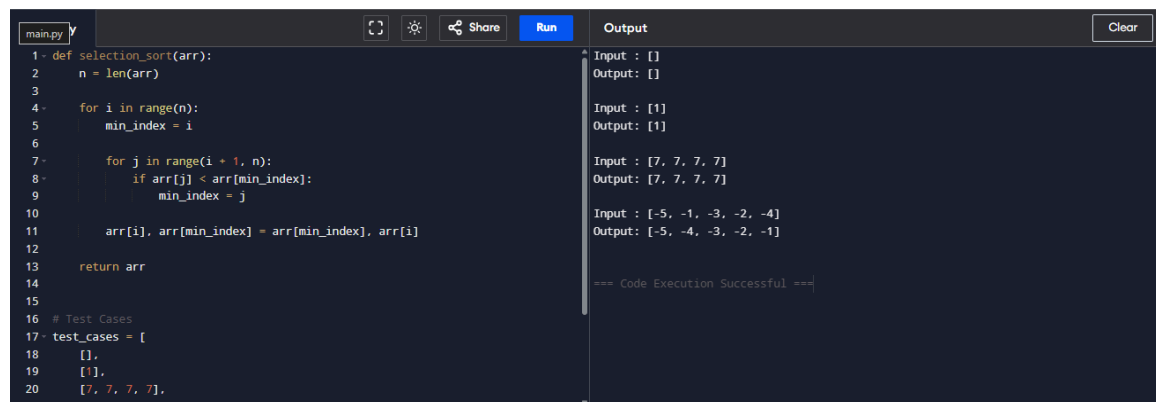
[1]

Test Case 3:

[7, 7, 7, 7]

Test Case 4:

[-5, -4, -3, -2, -1]



The screenshot shows a code editor with a dark theme. The left pane contains Python code for a selection sort function and test cases. The right pane shows the output of the code execution for each test case. The code defines a `selection_sort(arr)` function that sorts an array in ascending order. It then defines a list of test cases: `test_cases = [[], [1], [7, 7, 7, 7], [-5, -4, -3, -2, -1]]`. The output pane shows the input and output for each test case, confirming that the function works correctly for all cases. The output for the last test case is `[-5, -4, -3, -2, -1]`.

```
1- def selection_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         min_index = i
6-
7-         for j in range(i + 1, n):
8-             if arr[j] < arr[min_index]:
9-                 min_index = j
10-
11-         arr[i], arr[min_index] = arr[min_index], arr[i]
12-
13-     return arr
14-
15-
16- # Test Cases
17- test_cases = [
18-     [],
19-     [1],
20-     [7, 7, 7, 7],
21-     [-5, -4, -3, -2, -1]
```

Input : []
Output : []

Input : [1]
Output : [1]

Input : [7, 7, 7, 7]
Output : [7, 7, 7, 7]

Input : [-5, -4, -3, -2, -1]
Output : [-5, -4, -3, -2, -1]

=== Code Execution Successful ===

Result:

The program was executed successfully and the expected output was obtained.

Experiment 2

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        min_index = i
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
    arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
    return arr
```

```
# Test Cases
```

```
arrays = [
```

```
    [5, 2, 9, 1, 5, 6],
```

```
    [10, 8, 6, 4, 2],
```

```
[1, 2, 3, 4, 5]  
]
```

```
for arr in arrays:
```

```
    print("Input :", arr)
```

```
    print("Sorted:", selection_sort(arr.copy()))
```

```
    print()
```

Sample Input:

Test Case 1:

```
[5, 2, 9, 1, 5, 6]
```

Test Case 2:

```
[10, 8, 6, 4, 2]
```

Test Case 3:

```
[1, 2, 3, 4, 5]
```

Sample Output:

Test Case 1:

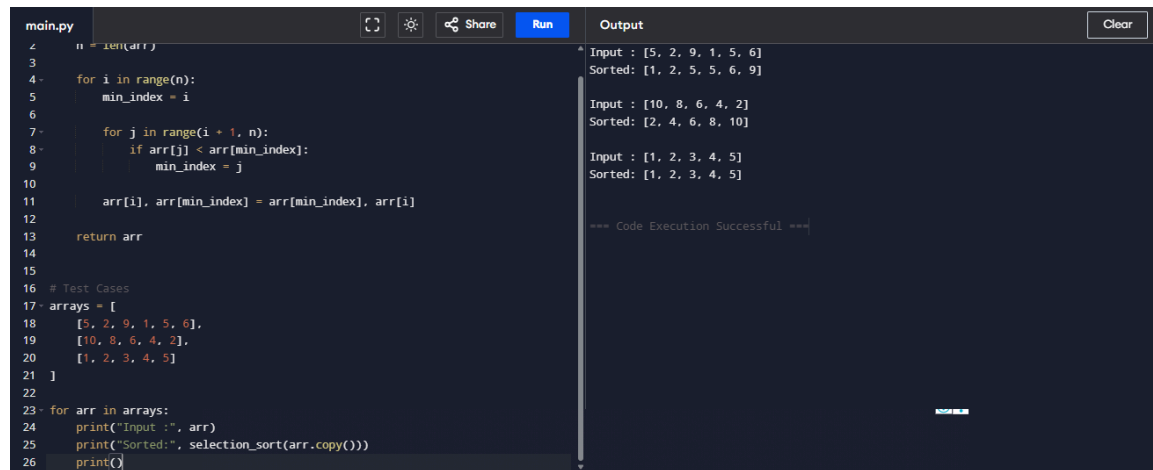
```
[1, 2, 5, 5, 6, 9]
```

Test Case 2:

```
[2, 4, 6, 8, 10]
```

Test Case 3:

```
[1, 2, 3, 4, 5]
```



The screenshot shows a code editor with a file named 'main.py'. The code implements a selection sort algorithm. It defines a function 'selection_sort(arr)' that iterates through the array, finding the minimum element in the unsorted portion and swapping it with the current element. Below the function, there are test cases: a list of arrays and a loop that prints the input and sorted output for each. The 'Output' pane on the right shows the results of running the code, displaying the input and sorted arrays for each test case, followed by a success message.

```
main.py
4 n = len(arr)
5 for i in range(n):
6     min_index = i
7     for j in range(i + 1, n):
8         if arr[j] < arr[min_index]:
9             min_index = j
10    arr[i], arr[min_index] = arr[min_index], arr[i]
11
12    return arr
13
14 # Test Cases
15 arrays = [
16     [5, 2, 9, 1, 5, 6],
17     [10, 8, 6, 4, 2],
18     [1, 2, 3, 4, 5]
19 ]
20
21 for arr in arrays:
22     print("Input :", arr)
23     print("Sorted:", selection_sort(arr.copy()))
24     print()
```

Output

```
Input : [5, 2, 9, 1, 5, 6]
Sorted: [1, 2, 5, 5, 6, 9]

Input : [10, 8, 6, 4, 2]
Sorted: [2, 4, 6, 8, 10]

Input : [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 3

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        swapped = False
```

```
        for j in range(n - i - 1):
```

```
            if arr[j] > arr[j + 1]:
```

```
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
                swapped = True
```

```
        if not swapped:
```

```
            break
```

```
    return arr
```

Test Cases

```
tests = [
```

```
    [64, 25, 12, 22, 11],
```

```
    [29, 10, 14, 37, 13],
```

```
[3, 5, 2, 1, 4],  
[1, 2, 3, 4, 5],  
[5, 4, 3, 2, 1]  
]
```

```
for arr in tests:
```

```
    print("Input :", arr)  
    print("Sorted:", bubble_sort(arr.copy()))  
    print()
```

Sample Input:

Test Case 1:

```
[64, 25, 12, 22, 11]
```

Test Case 2:

```
[29, 10, 14, 37, 13]
```

Test Case 3:

```
[3, 5, 2, 1, 4]
```

Test Case 4:

```
[1, 2, 3, 4, 5]
```

Test Case 5:

```
[5, 4, 3, 2, 1]
```


Sample Output:

Test Case 1:

[11, 12, 22, 25, 64]

Test Case 2:

[10, 13, 14, 29, 37]

Test Case 3:

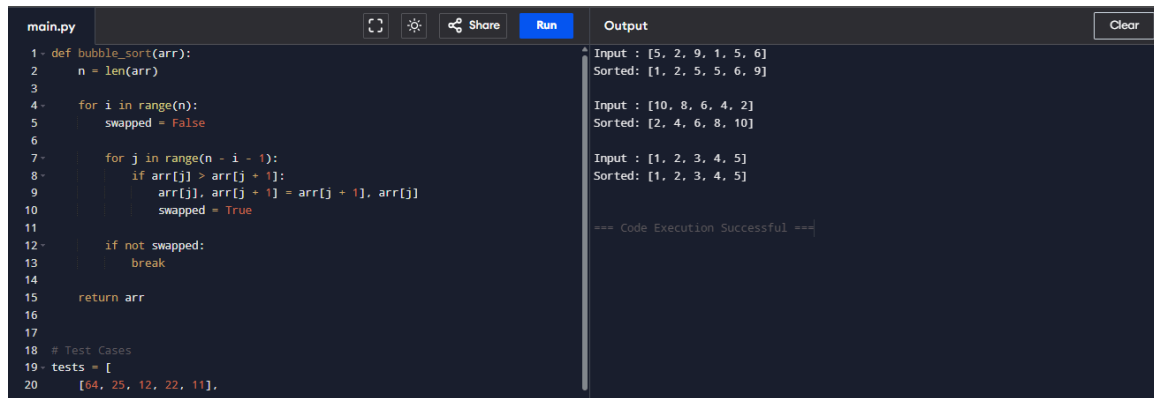
[1, 2, 3, 4, 5]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[1, 2, 3, 4, 5]



The screenshot shows a code editor with a file named 'main.py'. The code implements a bubble sort algorithm. It defines a function 'bubble_sort(arr)' that takes an array and returns the sorted array. The code includes test cases for various input arrays. The output panel shows the results of these test cases, confirming that the arrays are sorted correctly. The code execution was successful.

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(n - i - 1):
8             if arr[j] > arr[j + 1]:
9                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
10                swapped = True
11
12        if not swapped:
13            break
14
15    return arr
16
17
18 # Test Cases
19 tests = [
20     [64, 25, 12, 22, 11],
21     [10, 13, 14, 29, 37],
22     [1, 2, 3, 4, 5],
23     [1, 2, 3, 4, 5],
24     [1, 2, 3, 4, 5]
25 ]
```

Output:

```
Input : [5, 2, 9, 1, 5, 6]
Sorted: [1, 2, 5, 5, 6, 9]

Input : [10, 8, 6, 4, 2]
Sorted: [2, 4, 6, 8, 10]

Input : [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 4

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.

2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def insertion_sort(arr):
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
```

```
            arr[j + 1] = arr[j]
```

```
            j -= 1
```

```
        arr[j + 1] = key
```

```
    return arr
```

Test Cases

```
arrays = [
```

```
    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3],
```

```
    [5, 5, 5, 5, 5],
```

```
    [2, 3, 1, 3, 2, 1, 1, 3]
```

```
]
```

```
for arr in arrays:
```

```
    print("Input :", arr)
```

```
    print("Sorted:", insertion_sort(arr.copy()))
```

```
    print()
```

Sample Input:

Test Case 1:

[64, 25, 12, 22, 11]

Test Case 2:

[29, 10, 14, 37, 13]

Test Case 3:

[3, 5, 2, 1, 4]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[5, 4, 3, 2, 1]

Sample Output:

Test Case 1:

[11, 12, 22, 25, 64]

Test Case 2:

[10, 13, 14, 29, 37]

Test Case 3:

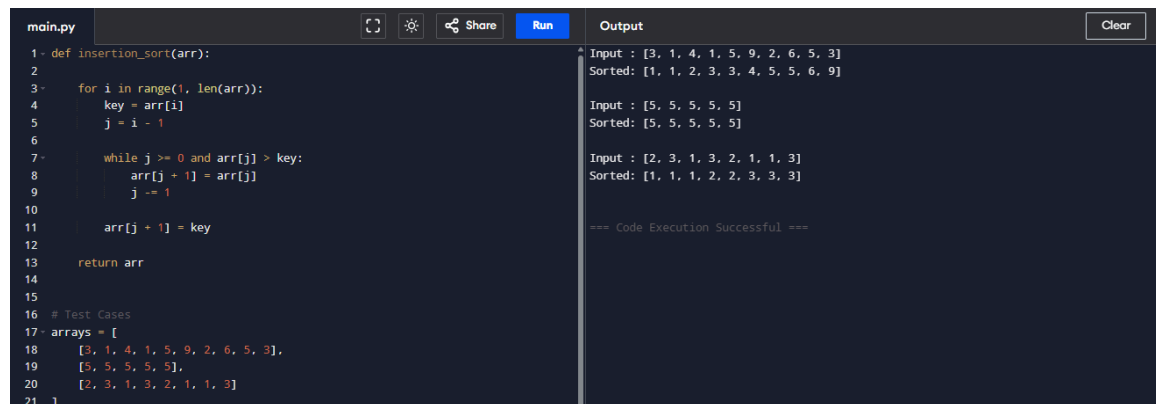
[1, 2, 3, 4, 5]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[1, 2, 3, 4, 5]



The screenshot shows a code editor with a file named 'main.py'. The code implements an insertion sort algorithm. The 'Run' button is highlighted in blue. The 'Output' panel on the right shows the results of three test cases. The first test case input is [3, 1, 4, 1, 5, 9, 2, 6, 5, 3] and the sorted output is [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]. The second test case input is [5, 5, 5, 5, 5] and the sorted output is [5, 5, 5, 5, 5]. The third test case input is [2, 3, 1, 3, 2, 1, 1, 3] and the sorted output is [1, 1, 1, 2, 2, 3, 3, 3]. The code execution was successful.

```
main.py [ ] [ ] [ ] Share Run Output Clear
1- def insertion_sort(arr):
2
3-     for i in range(1, len(arr)):
4         key = arr[i]
5         j = i - 1
6
7-         while j >= 0 and arr[j] > key:
8             arr[j + 1] = arr[j]
9             j -= 1
10
11        arr[j + 1] = key
12
13    return arr
14
15
16 # Test Cases
17 arrays = [
18     [3, 1, 4, 1, 5, 9, 2, 6, 5, 3],
19     [5, 5, 5, 5, 5],
20     [2, 3, 1, 3, 2, 1, 1, 3]
21 ]
```

Input : [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Sorted: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input : [5, 5, 5, 5, 5]
Sorted: [5, 5, 5, 5, 5]

Input : [2, 3, 1, 3, 2, 1, 1, 3]
Sorted: [1, 1, 1, 2, 2, 3, 3, 3]

=== Code Execution Successful ===

Result:

The program was executed successfully and the expected output was obtained.

Experiment 5

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def findKthPositive(arr, k):
```

```
    current = 1
```

```
    index = 0
```

```
    while k > 0:
```

```
        if index < len(arr) and arr[index] == current:
```

```
            index += 1
```

```
        else:
```

```
            k -= 1
```

```
            if k == 0:
```

```
                return current
```

```
            current += 1
```

```
# Test Cases
```

```
print("Example 1")
```

```
arr = [2, 3, 4, 7, 11]
```

```
k = 5
```

```
print("Output:", findKthPositive(arr, k))
```

```
print()
```

```
print("Example 2")
```

```
arr = [1, 2, 3, 4]
```

```
k = 2
```

```
print("Output:", findKthPositive(arr, k))
```

Sample Input:

Test Case 1:

Array = [2, 3, 4, 7, 11]

K = 5

Test Case 2:

Array = [1, 2, 3, 4]

K = 2

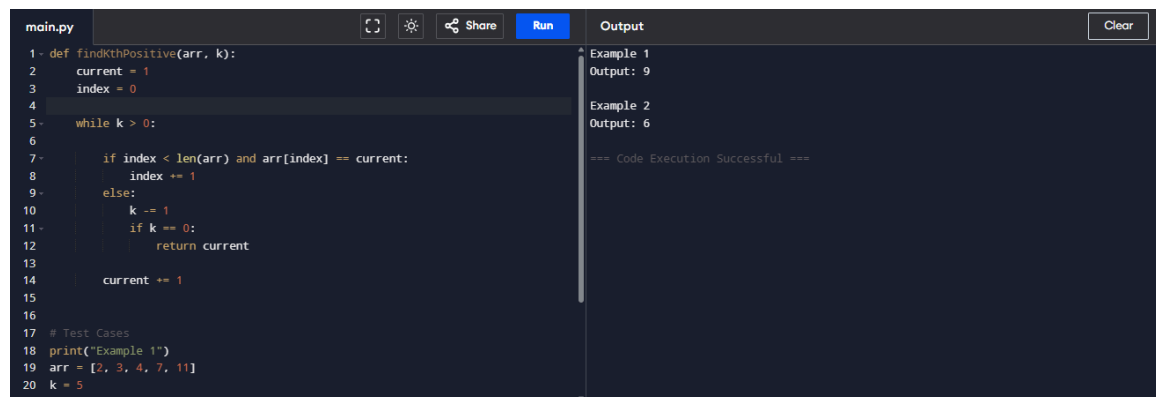
Sample Output:

Test Case 1:

9

Test Case 2:

6



The screenshot shows a code editor with a dark theme. The left pane displays a Python file named 'main.py' with the following code:

```
1- def findKthPositive(arr, k):
2-     current = 1
3-     index = 0
4-
5-     while k > 0:
6-
7-         if index < len(arr) and arr[index] == current:
8-             index += 1
9-         else:
10-            k -= 1
11-            if k == 0:
12-                return current
13-
14-            current += 1
15-
16-
17- # Test Cases
18- print("Example 1")
19- arr = [2, 3, 4, 7, 11]
20- k = 5
21- print("Output: ", findKthPositive(arr, k))
```

The right pane shows the output of the program:

```
Example 1
Output: 9

Example 2
Output: 6

=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 6

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def brute_force(text, pattern):
```

```
    n = len(text)
```

```
    m = len(pattern)
```

```
    for i in range(n - m + 1):
```

```
        j = 0
```

```
        while j < m and text[i + j] == pattern[j]:
```

```
            j += 1
```

```
        if j == m:
```

```
            return i
```

```
    return -1
```

```
text = input("Enter Text: ")
```

```
pattern = input("Enter Pattern: ")
```

```
index = brute_force(text, pattern)
```

```
if index != -1:
```

```
print("Pattern found at index", index)
```

else:

```
print("Pattern not found")
```

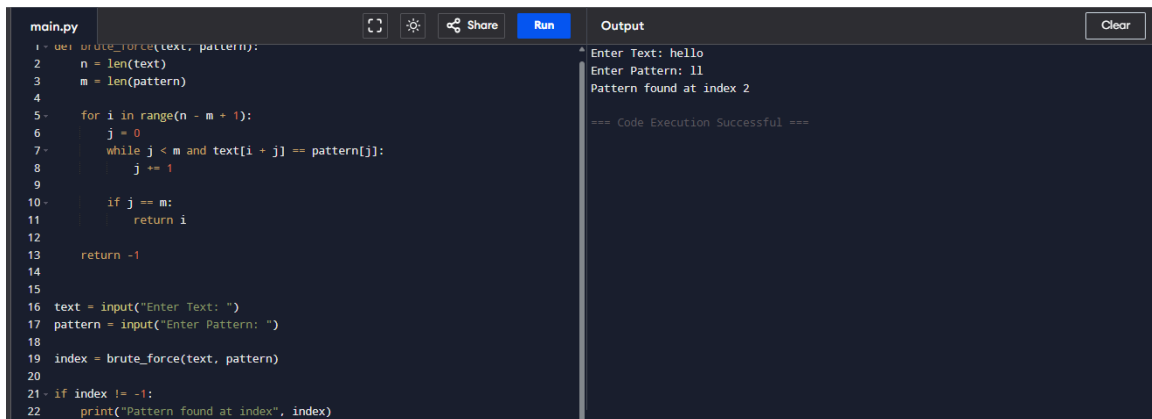
Sample Input:

Enter Text: AABAACAADAABAABA

Enter Pattern: AABA

Sample Output:

Pattern found at index 0



The screenshot shows a code editor with a file named 'main.py'. The code implements a brute-force algorithm to find a pattern in a text. It defines a function 'brute_force(text, pattern)' that returns the starting index of the pattern or -1 if not found. The main part of the script takes user input for 'text' and 'pattern', calls the function, and prints the result. The output pane on the right shows the execution with inputs 'hello' and 'll', resulting in 'Pattern found at index 2'. A success message '=== Code Execution Successful ===' is also displayed.

```
1 def brute_force(text, pattern):
2     n = len(text)
3     m = len(pattern)
4
5     for i in range(n - m + 1):
6         j = 0
7         while j < m and text[i + j] == pattern[j]:
8             j += 1
9
10        if j == m:
11            return i
12
13    return -1
14
15
16 text = input("Enter Text: ")
17 pattern = input("Enter Pattern: ")
18
19 index = brute_force(text, pattern)
20
21 if index != -1:
22     print("Pattern found at index", index)
```

Output

```
Enter Text: hello
Enter Pattern: ll
Pattern found at index 2

=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 7

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
import math
```

```
n = int(input("Enter number of points: "))
```

```
points = []
```

```
for i in range(n):
```

```
    x, y = map(int, input().split())
```

```
    points.append((x, y))
```

```
min_dist = float('inf')
```

```
pair = ()
```

```
for i in range(n):
```

```
    for j in range(i + 1, n):
```

```
        d = math.sqrt((points[i][0] - points[j][0]) ** 2 +
```

```
                      (points[i][1] - points[j][1]) ** 2)
```

```
        if d < min_dist:
```

```
            min_dist = d
```

```
            pair = (points[i], points[j])
```

```
print("Closest Pair:", pair)
```

```
print("Minimum Distance:", round(min_dist, 2))
```

Sample Input:

Enter number of points:

4

2 3

12 30

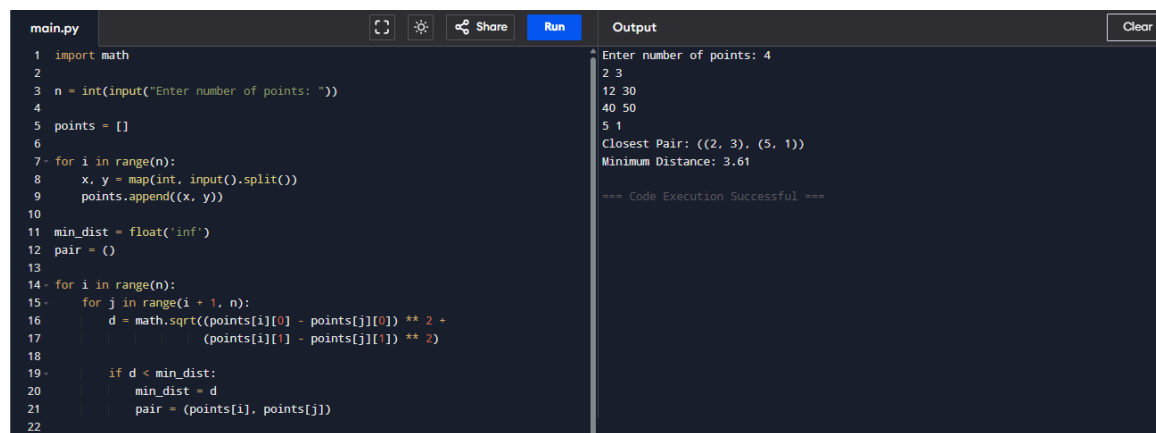
40 50

5 1

Sample Output:

Closest Pair: ((2, 3), (5, 1))

Minimum Distance: 3.61



The screenshot shows a code editor with a file named 'main.py'. The code is a Python program that reads a number of points, then reads pairs of coordinates, and finds the closest pair. The output window shows the execution results.

```
main.py
1 import math
2
3 n = int(input("Enter number of points: "))
4
5 points = []
6
7 for i in range(n):
8     x, y = map(int, input().split())
9     points.append((x, y))
10
11 min_dist = float('inf')
12 pair = ()
13
14 for i in range(n):
15     for j in range(i + 1, n):
16         d = math.sqrt((points[i][0] - points[j][0]) ** 2 +
17                       (points[i][1] - points[j][1]) ** 2)
18
19         if d < min_dist:
20             min_dist = d
21             pair = (points[i], points[j])
22
```

Output

```
Enter number of points: 4
2 3
12 30
40 50
5 1
Closest Pair: ((2, 3), (5, 1))
Minimum Distance: 3.61

=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 8

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input
Process using algorithm
Print output
STOP

Python Program:
from itertools import permutations

```
graph = [  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
]
```

```
n = len(graph)  
vertices = list(range(1, n))  
min_cost = float('inf')  
for path in permutations(vertices):  
    cost = 0  
    k = 0  
  
    for j in path:  
        cost += graph[k][j]  
        k = j  
    cost += graph[k][0]  
    min_cost = min(min_cost, cost)  
print("Minimum Cost:", min_cost)
```

Sample Input:
Distance Matrix

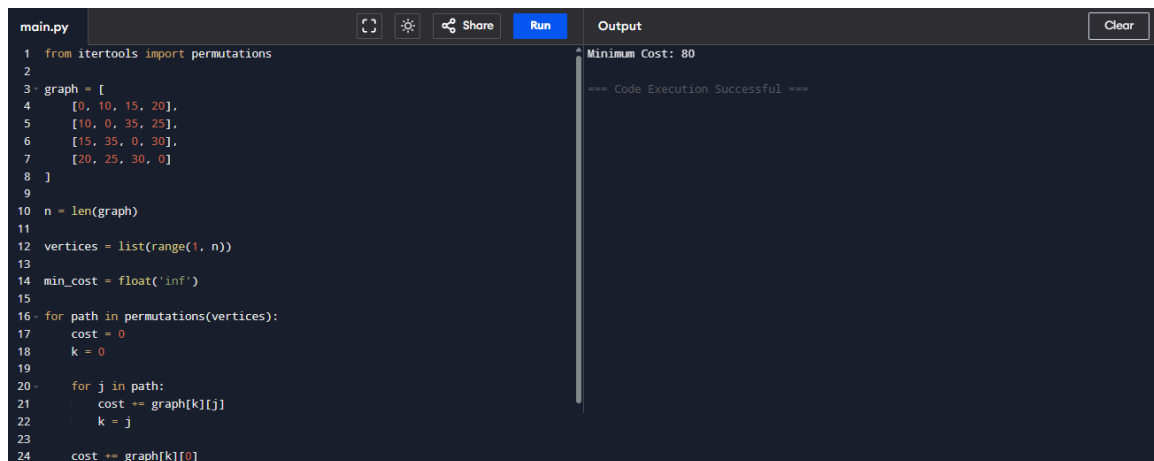
0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Sample Output:
Minimum Cost: 80



The screenshot shows a code editor with a file named 'main.py'. The code defines a graph with 4 vertices and calculates the minimum cost by checking all possible paths. The output window shows 'Minimum Cost: 80' and '=== Code Execution Successful ==='.

```
1 from itertools import permutations
2
3 graph = [
4     [0, 10, 15, 20],
5     [10, 0, 35, 25],
6     [15, 35, 0, 30],
7     [20, 25, 30, 0]
8 ]
9
10 n = len(graph)
11
12 vertices = list(range(1, n))
13
14 min_cost = float('inf')
15
16 for path in permutations(vertices):
17     cost = 0
18     k = 0
19
20     for j in path:
21         cost += graph[k][j]
22         k = j
23
24     cost += graph[k][0]
```

Output: Minimum Cost: 80
=== Code Execution Successful ===

Result:
The program was executed successfully and the expected output was obtained.

Experiment 9

Aim:
As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START
Read input
Process using algorithm

Print output
STOP

Python Program:

```
def orientation(p, q, r):  
    return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
```

```
def convex_hull(points):
```

```
    if len(points) < 3:
```

```
        return []
```

```
    left = 0
```

```
    for i in range(1, len(points)):
```

```
        if points[i][0] < points[left][0]:
```

```
            left = i
```

```
    hull = []
```

```
    p = left
```

```
    while True:
```

```
        hull.append(points[p])
```

```
        q = (p + 1) % len(points)
```

```
        for i in range(len(points)):
```

```
            if orientation(points[p], points[i], points[q]) < 0:
```

```
                q = i
```

```
p = q
```

```
if p == left:
```

```
    break
```

```
return hull
```

```
points = [(0,3),(2,2),(1,1),(2,1),(3,0),(0,0),(3,3)]
```

```
print("Convex Hull:")
```

```
print(convex_hull(points))
```

Sample Input:

Points:

(0,3)

(2,2)

(1,1)

(2,1)

(3,0)

(0,0)

(3,3)

Sample Output:

Convex Hull:

[(0, 3), (0, 0), (3, 0), (3, 3)]

Python Program:

```
from itertools import permutations
```

```
cost = [  
    [9,2,7],  
    [6,4,3],  
    [5,8,1]  
]
```

```
n = len(cost)
```

```
workers = range(n)
```

```
min_cost = float('inf')
```

```
best = None
```

```
for perm in permutations(workers):
```

```
    total = 0
```

```
    for job in workers:
```

```
        total += cost[perm[job]][job]
```

```
    if total < min_cost:
```

```
        min_cost = total
```

```
        best = perm
```

```
print("Minimum Cost:", min_cost)
```

```
print("Assignment:", best)
```

Sample Input:

Cost Matrix

9 2 7

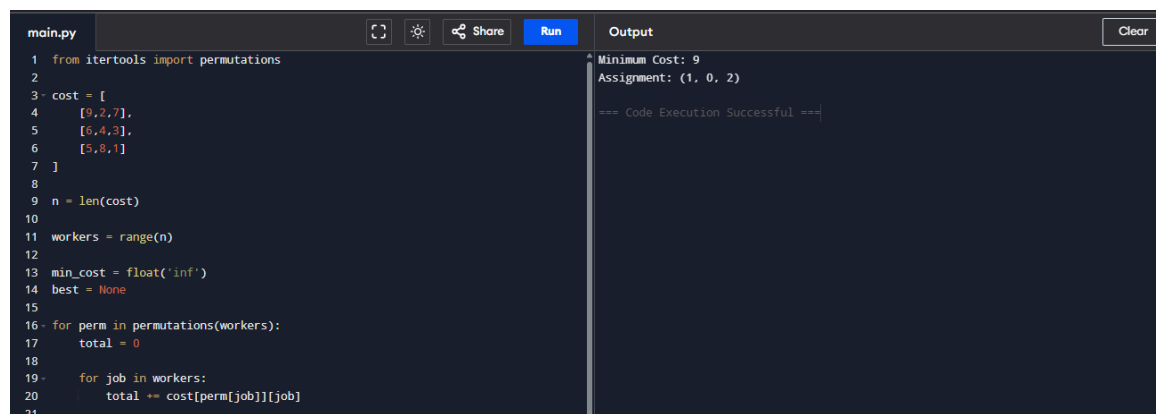
6 4 3

5 8 1

Sample Output:

Minimum Cost: 9

Assignment: (1, 0, 2)



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 from itertools import permutations
2
3 cost = [
4     [9,2,7],
5     [6,4,3],
6     [5,8,1]
7 ]
8
9 n = len(cost)
10
11 workers = range(n)
12
13 min_cost = float('inf')
14 best = None
15
16 for perm in permutations(workers):
17     total = 0
18     for job in workers:
19         total += cost[perm[job]][job]
20
21
```

The output panel on the right shows the following text:

```
Minimum Cost: 9
Assignment: (1, 0, 2)
=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 11

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
def knapsack(weights, values, capacity):  
    n = len(values)  
  
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]  
  
    for i in range(1, n + 1):  
        for w in range(1, capacity + 1):  
            if weights[i - 1] <= w:  
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],  
                               dp[i - 1][w])  
            else:  
                dp[i][w] = dp[i - 1][w]  
        return dp[n][capacity]  
weights = [2, 3, 4, 5]  
values = [3, 4, 5, 6]  
capacity = 5  
print("Weights :", weights)  
print("Values :", values)  
print("Capacity:", capacity)  
print("Maximum Profit:", knapsack(weights, values, capacity))
```

Sample Input:

Weights = [2, 3, 4, 5]

Values = [3, 4, 5, 6]

Capacity = 5

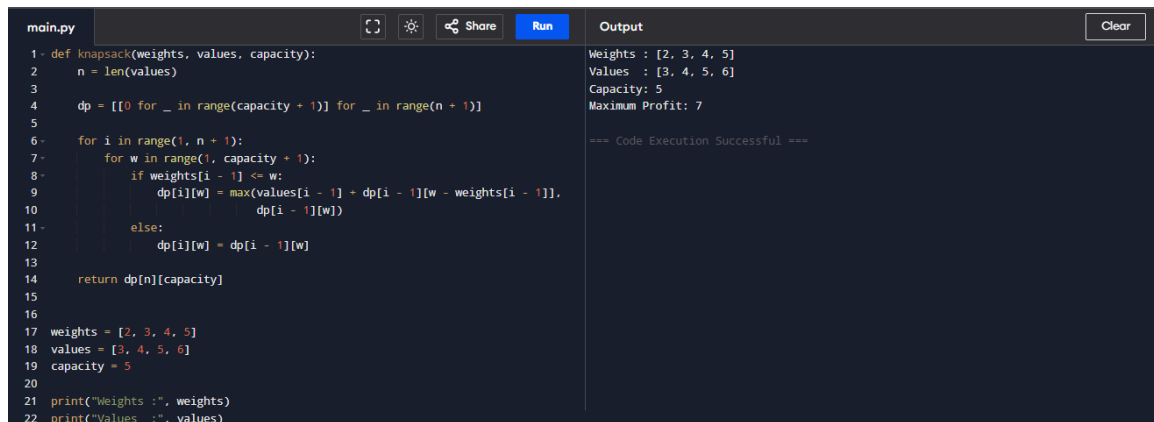
Sample Output:

Weights : [2, 3, 4, 5]

Values : [3, 4, 5, 6]

Capacity: 5

Maximum Profit: 7



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'knapsack' that takes 'weights', 'values', and 'capacity' as arguments. It calculates the maximum profit using a dynamic programming approach. The main part of the code sets 'weights' to [2, 3, 4, 5], 'values' to [3, 4, 5, 6], and 'capacity' to 5. It then prints these values and the result of the 'knapsack' function. The output panel on the right shows the execution results: 'Weights : [2, 3, 4, 5]', 'Values : [3, 4, 5, 6]', 'Capacity: 5', and 'Maximum Profit: 7'. A message at the bottom of the output panel states '=== Code Execution Successful ==='.

```
main.py
1 def knapsack(weights, values, capacity):
2     n = len(values)
3
4     dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
5
6     for i in range(1, n + 1):
7         for w in range(1, capacity + 1):
8             if weights[i - 1] <= w:
9                 dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
10                                dp[i - 1][w])
11             else:
12                 dp[i][w] = dp[i - 1][w]
13
14     return dp[n][capacity]
15
16
17 weights = [2, 3, 4, 5]
18 values = [3, 4, 5, 6]
19 capacity = 5
20
21 print("Weights :", weights)
22 print("Values :", values)
```

Output

Weights : [2, 3, 4, 5]
Values : [3, 4, 5, 6]
Capacity: 5
Maximum Profit: 7

=== Code Execution Successful ===

Result:

The program was executed successfully and the expected output was obtained.

Experiment 12

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
items = [  
    (60, 10),  
    (100, 20),  
    (120, 30)  
]  
  
capacity = 50  
  
items.sort(key=lambda x: x[0] / x[1], reverse=True)  
  
profit = 0  
  
for value, weight in items:  
    if capacity >= weight:  
        profit += value  
        capacity -= weight  
    else:  
        profit += value * (capacity / weight)  
        break  
  
print("Items (Value, Weight):", items)  
print("Maximum Profit:", profit)
```

Sample Input:

Items:

(60,10)

(100,20)

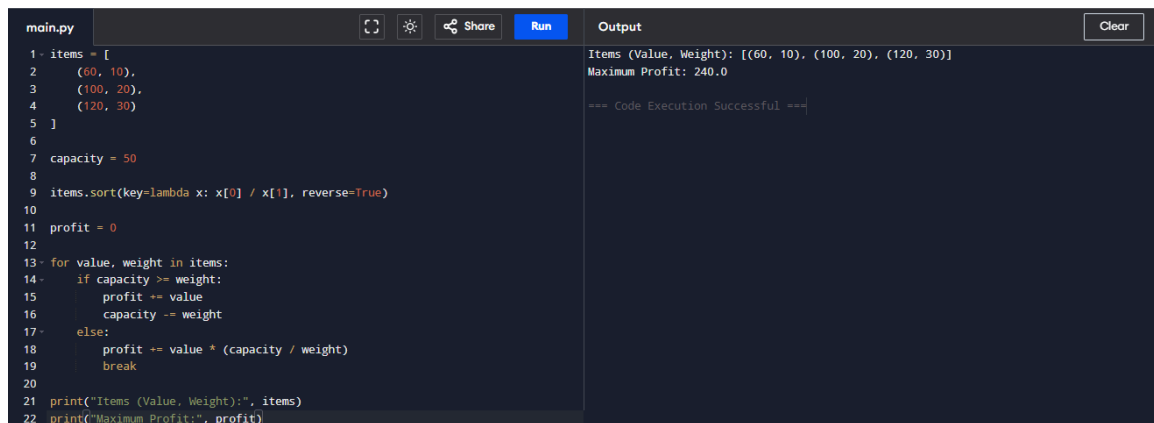
(120,30)

Capacity = 50

Sample Output:

Items (Value, Weight): [(60, 10), (100, 20), (120, 30)]

Maximum Profit: 240.0



The screenshot shows a code editor with a file named 'main.py'. The code defines a list of items with their values and weights, sets a capacity of 50, sorts the items by value-to-weight ratio, and then iterates through them to calculate the maximum profit. The output panel shows the items list and the maximum profit of 240.0.

```
1 items = [  
2     (60, 10),  
3     (100, 20),  
4     (120, 30)  
5 ]  
6  
7 capacity = 50  
8  
9 items.sort(key=lambda x: x[0] / x[1], reverse=True)  
10  
11 profit = 0  
12  
13 for value, weight in items:  
14     if capacity >= weight:  
15         profit += value  
16         capacity -= weight  
17     else:  
18         profit += value * (capacity / weight)  
19         break  
20  
21 print("Items (Value, Weight):", items)  
22 print("Maximum Profit:", profit)
```

Output

```
Items (Value, Weight): [(60, 10), (100, 20), (120, 30)]  
Maximum Profit: 240.0  
  
=== Code Execution Successful ===
```

Result:

The program was executed successfully and the expected output was obtained.

Experiment 13

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

```
import heapq

text = "AAABBCCCCDD"

frequency = {}

for ch in text:

    frequency[ch] = frequency.get(ch, 0) + 1

heap = [[weight, [char, ""]] for char, weight in frequency.items()]

heapq.heapify(heap)

while len(heap) > 1:

    low = heapq.heappop(heap)

    high = heapq.heappop(heap)

    for pair in low[1:]:

        pair[1] = '0' + pair[1]

    for pair in high[1:]:

        pair[1] = '1' + pair[1]

    heapq.heappush(heap, [low[0] + high[0]] + low[1:] + high[1:])

codes = sorted(heapq.heappop(heap)[1:], key=lambda p: (len(p[1]), p))

print("Character\tCode")

for char, code in codes:

    print(char, "\t\t", code)
```

Sample Input:

Text = AAABBCCCCDD

Sample Output:

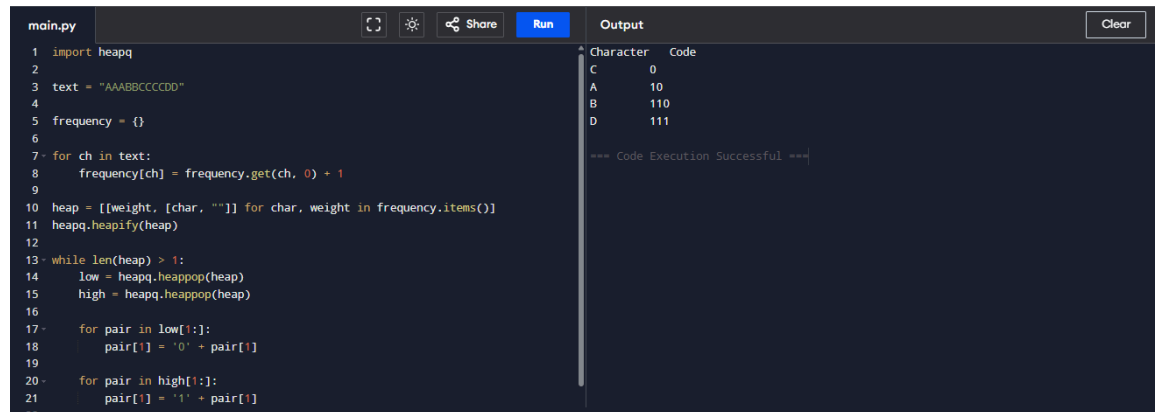
Character Code

C 0

A 10

B 110

D 111



The screenshot shows a code editor with a file named 'main.py'. The code implements a Huffman coding algorithm. It takes the input string 'AABBBCCCCDD', calculates the frequency of each character, and then builds a binary tree using a min-heap. The resulting binary codes are: C (0), A (10), B (110), and D (111). The output window on the right displays these results in a table and confirms that the code execution was successful.

```
1 import heapq
2
3 text = "AABBBCCCCDD"
4
5 frequency = {}
6
7 for ch in text:
8     frequency[ch] = frequency.get(ch, 0) + 1
9
10 heap = [[weight, [char, ""]] for char, weight in frequency.items()]
11 heapq.heapify(heap)
12
13 while len(heap) > 1:
14     low = heapq.heappop(heap)
15     high = heapq.heappop(heap)
16
17     for pair in low[1:]:
18         pair[1] = '0' + pair[1]
19
20     for pair in high[1:]:
21         pair[1] = '1' + pair[1]
```

Character	Code
C	0
A	10
B	110
D	111

=== Code Execution Successful ===

Result:

The program was executed successfully and the expected output was obtained.

Experiment 14

Aim:

As specified in the uploaded experiment sheet.

Algorithm:

1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:

START

Read input

Process using algorithm

Print output

STOP

Python Program:

N = 4

```
board = [[0] * N for _ in range(N)]
```

```
def is_safe(row, col):
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```
i, j = row, col
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j]:
```

```
        return False
```

```
    i -= 1
```

```
    j -= 1
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j]:
```

```
        return False
```

```
    i += 1
```

```
    j -= 1
```

```
return True
```



```
def solve(col):  
    if col >= N:  
        return True  
  
    for i in range(N):  
        if is_safe(i, col):  
            board[i][col] = 1  
  
            if solve(col + 1):  
                return True  
  
            board[i][col] = 0  
  
    return False  
  
solve(0)  
  
print("Solution for", N, "Queens:\n")  
  
for row in board:  
    print(row)
```

Sample Input:

Number of Queens = 4

Sample Output:

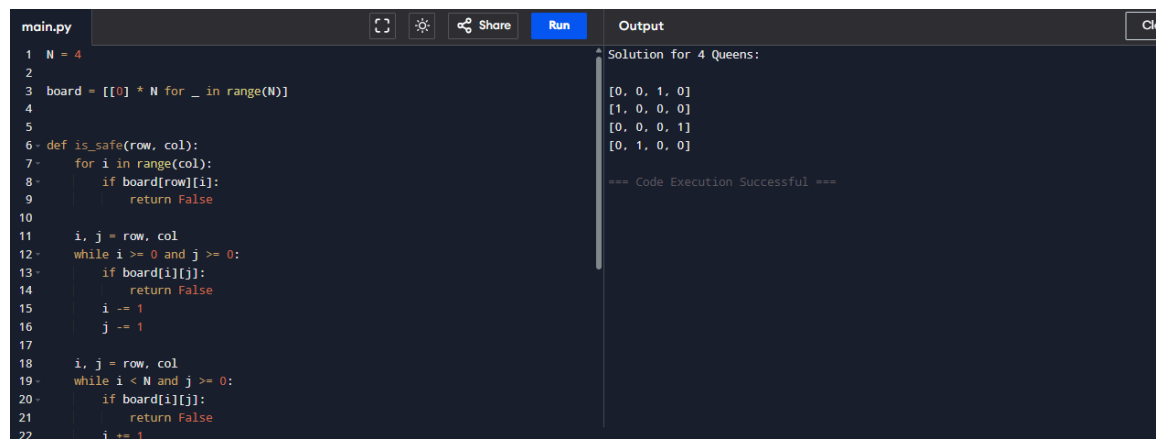
Solution for 4 Queens:

[0, 0, 1, 0]

[1, 0, 0, 0]

[0, 0, 0, 1]

[0, 1, 0, 0]



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'is_safe' to check if a queen can be placed at a given row and column, and then iterates through columns for each row to find a valid position. The output window displays the solution for 4 queens as a list of four rows, each representing the column index of the queen in that row.

```
main.py
1 N = 4
2
3 board = [[0] * N for _ in range(N)]
4
5
6 def is_safe(row, col):
7     for i in range(col):
8         if board[row][i]:
9             return False
10
11     i, j = row, col
12     while i >= 0 and j >= 0:
13         if board[i][j]:
14             return False
15         i -= 1
16         j -= 1
17
18     i, j = row, col
19     while i < N and j >= 0:
20         if board[i][j]:
21             return False
22         i += 1
```

Output

Solution for 4 Queens:

```
[0, 0, 1, 0]
[1, 0, 0, 0]
[0, 0, 0, 1]
[0, 1, 0, 0]
```

=== Code Execution Successful ===

Result:

The program was executed successfully and the expected output was obtained.